

Universidad Nacional Autónoma de México

Facultad de Ciencias

Ciencias de la Computación

---

# Práctica 4: Árbol de Sintaxis Abstracta (AST)

Compiladores

Semestre 2027-1

**Equipo:** DFGR

**Integrantes:** Flores Galeana Daniela - 321162342  
Rivera Machuca Gabriel Eduardo - 321057608

**Profesora:** L. en C.C. Yessica Janeth Pablo Martínez

**Ayudante:** Pedro Yamil Salazar González

**Fecha de entrega:** 2 de octubre de 2026

# Índice

|                                                   |   |
|---------------------------------------------------|---|
| 1. Objetivo                                       | 2 |
| 2. Evolución respecto de la Práctica 3            | 2 |
| 2.1. Módulos agregados y modificados . . . . .    | 2 |
| 3. Diseño del AST                                 | 2 |
| 4. Construcción desde el parser                   | 2 |
| 5. Propiedad y memoria                            | 3 |
| 6. Impresión canónica                             | 3 |
| 7. Diagnósticos y recuperación                    | 3 |
| 8. Resultados y pruebas                           | 3 |
| 9. Problemas encontrados                          | 4 |
| 10.División del trabajo                           | 4 |
| 11.Uso de herramientas de Inteligencia Artificial | 4 |
| 12.Conclusiones                                   | 4 |
| 13.Referencias                                    | 5 |

# 1. Objetivo

El objetivo principal de esta cuarta práctica fue extender el analizador sintáctico descendente recursivo del lenguaje MiniC desarrollado en la entrega anterior para construir un **Árbol de Sintaxis Abstracta (AST)**. A diferencia del análisis sintáctico puro que únicamente valida la corrección gramatical, el AST representa de manera jerárquica y esencial la estructura lógica del programa, descartando elementos puramente sintácticos o de puntuación (como paréntesis de agrupación y puntos y coma) y sirviendo como la representación intermedia fundamental para las etapas posteriores de análisis semántico y generación de código.

# 2. Evolución respecto de la Práctica 3

## 2.1. Módulos agregados y modificados

Se incorporó un nuevo módulo dedicado por completo al manejo del AST, estructurado en `src/ast/ast.c` y `include/ast/ast.h`. Las funciones del parser (`parser.c`) fueron refactorizadas de modo que dejaron de retornar `void` para convertirse en funciones constructoras que devuelven nodos del tipo `ASTNode *`. Se conservaron intactas las ventajas arquitectónicas previas: la interfaz incremental del lexer, el control de errores léxicos y la recuperación en modo pánico.

# 3. Diseño del AST

El diseño se estructuró mediante una representación modular basada en una unión centralizada dentro de la estructura `ASTNode`, permitiendo distinguir con claridad los nodos de programa, bloques, declaraciones, asignaciones, sentencias de control (`if`, `while`), expresiones binarias, unarias y literales.

- **Listas dinámicas:** Se implementó la estructura `ASTNodeList` para almacenar colecciones arbitrarias de sentencias en programas y bloques mediante un crecimiento dinámico basado en `realloc`.
- **Operadores:** Se definieron enumeraciones específicas para operadores binarios (`BinaryOperator`) y unarios (`UnaryOperator`).
- **Posicionamiento:** Cada nodo conserva de manera obligatoria la línea y columna exactas de su origen en el código fuente para facilitar futuros diagnósticos semánticos.

# 4. Construcción desde el parser

Cada función de producción del parser fue adaptada para crear nodos hijos y enlazarlos de manera ascendente. Las funciones de expresiones se estructuraron respetando

estrictamente la jerarquía de precedencia y asociatividad (izquierda para operadores aditivos, multiplicativos y lógicos; derecha para el operador unario negativo), garantizando que operadores de mayor prioridad descendan correctamente como subárboles de los de menor prioridad.

## 5. Propiedad y memoria

Se estableció una política estricta de propiedad de recursos para evitar fugas de memoria y liberaciones dobles:

- El AST almacena copias independientes de cadenas e identificadores mediante `strdup`.
- Cada nodo es propietario exclusivo de sus hijos y de sus listas asociadas.
- Si un constructor o inserción falla, la responsabilidad de liberación recae temporalmente sobre el llamador hasta que se consolide un subárbol válido.
- La función `ast_destroy` realiza una liberación recursiva segura de todo el árbol a partir de la raíz.

## 6. Impresión canónica

Se implementó la función `ast_print` junto con un recorrido recursivo indentado que emite exactamente dos espacios por nivel de profundidad. La salida refleja fielmente las etiquetas canónicas publicadas (como `Program`, `VariableDeclaration`, `Assignment`, `Add`, `Identifier`, entre otras), omitiendo nodos estructurales intermedios como paréntesis y puntos y coma.

## 7. Diagnósticos y recuperación

El compilador mantiene el mecanismo de recuperación ante errores en modo pánico de la Práctica 3. No obstante, ante la presencia de cualquier error léxico o sintáctico, se aborta la construcción del árbol, se destruyen los nodos parciales creados, se omite la impresión del AST en `stdout` y el programa termina con un código de salida distinto de cero, asegurando que ningún AST corrupto sea propagado.

## 8. Resultados y pruebas

El sistema fue evaluado con la batería completa de pruebas públicas proporcionadas (de `p01` a `p12`), así como con pruebas unitarias propias en la carpeta `tests/equipo/`. El compilador superó exitosamente todas las validaciones de estructura del AST, precedencia, asociatividad y códigos de salida (0 para programas válidos y 1 para entradas

con errores). Asimismo, se verificó mediante herramientas de depuración la ausencia total de fugas de memoria.

## 9. Problemas encontrados

Durante el desarrollo se presentaron complicaciones menores al sincronizar la asignación de lexemas con la liberación de tokens temporales en `parse_primary`, ya que un avance prematuro del puntero provocaba la pérdida de referencias al lexema original. Esto se solucionó realizando una copia anticipada con `strdup` antes de consumir el token. Asimismo, se ajustaron los casos base de destrucción en listas anidadas de bloques para prevenir accesos inválidos.

## 10. División del trabajo

- **Gabriel Eduardo Rivera Machuca:** Diseño e implementación del módulo de gestión del AST (`ast.c` / `ast.h`), listas dinámicas, funciones constructoras y refactorización del parser de expresiones.
- **Daniela Flores Galeana:** Implementación de constructores de sentencias de control y bloques en el parser, diseño de la impresión canónica, integración de pruebas automatizadas y verificación de memoria.

Ambos integrantes participaron activamente en la revisión de la arquitectura y la depuración del código fuente.

## 11. Uso de herramientas de Inteligencia Artificial

Para esta práctica se utilizó Gemini (Google) como apoyo de consulta técnica.

- **Propósito:** Consultar las directrices de diseño para uniones en C y la gestión de contratos de propiedad de memoria entre el parser y el AST.
- **Apoyo en el reporte:** Organización y estructura formal del documento de reporte de acuerdo con los lineamientos de la facultad.
- **Revisión y comprobación:** Todo el código y los algoritmos integrados fueron probados, depurados y verificados de manera manual en el entorno de prácticas.

## 12. Conclusiones

La implementación del Árbol de Sintaxis Abstracta consolidó la transición de un compilador puramente analítico a uno estructurado capaz de representar formalmente la semántica de un programa. El dominio de la gestión dinámica de memoria y el diseño de jerarquías de nodos en C establecen las bases indispensables para afrontar con éxito las siguientes etapas de análisis semántico y construcción de tablas de símbolos.

## 13. Referencias

### Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2da ed.). Pearson.
- [2] Fischer, C. N., Cytron, R. K., & LeBlanc, R. J. (2009). *Crafting a compiler*. Pearson.
- [3] Nystrom, R. (2021). *Crafting Interpreters*. Genever Benning.
- [4] Pablo Martínez, Y. J. (2026). *Guía de integración y especificaciones de la Práctica 4*. Facultad de Ciencias, UNAM.